

claude-windows / d4cf3fd6-b5bd-4ed1-9b57-88eea69333cd

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\d4cf3fd6-b5bd-4ed1-9b57-88eea69333cd\subagents\workflows\wf_8c732492-138\agent-ae958a1af2ef5ef54.jsonl`  
- SHA-256: `ae9693e387ee75fe1097ed81904143bd54ebd906767b5043c2f31e2ebd78dd3b`  
- Source modified: `2026-07-04T08:55:05+00:00`  
- Imported at: `2026-07-05T16:48:23+00:00`  
- project: `wf_8c732492-138`  
- session_id: `d4cf3fd6-b5bd-4ed1-9b57-88eea69333cd`
```

Transcript

1. user (2026-07-04T08:47:59.439Z)

You are fixing a code-review blocker on PR #468 (branch fix/drain-concurrency) of Khelsutra/badminton-highlight-indexer.
Your worktree: C:/Users/avidu/Projects/khelsutra-guru/wt-drain-concurrency (must be on fix/drain-concurrency at 8534077)

ENVIRONMENT (read carefully):

- Windows 11 box; use the PowerShell tool for shell commands (or Bash for POSIX one-liners). Global Python 3.13.13 has the full offline dep set (pytest 9.0.3 works).
- Repo: Khelsutra/badminton-highlight-indexer. You work in an EXISTING git worktree (path given below) that is already checked out on the PR branch at the PR head. Other sibling worktrees exist for other PRs ? do NOT touch them, do NOT touch the main checkout at C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer.
- FIRST: run "git branch --show-current" and "git log --oneline -1" in your worktree and confirm they match the branch+head stated below. If they do not match, STOP and report.
- Do NOT commit. Do NOT push. Do NOT run any gh/network write command. Leave your changes UNCOMMITTED in the worktree; the orchestrator reviews, commits, and pushes.
- MANDATORY verification gates before you report done (run from the worktree root, report exact results):
 1. FULL suite: python -m pytest tests/ -q -p no:cacheprovider (~2-5 min; NEVER run just the changed test file as your final gate ? the whole suite must be green; report exact "N passed, M skipped" counts and any failures verbatim)
 2. Lint: python -m ruff check . (CI pins ruff==0.15.16; if ruff is missing or a different major, python -m pip install ruff==0.15.16)
 3. Types: python -m mypy backend training (CI pins mypy==2.1.0; install if missing)
- Note: sibling agents run their own pytest suites concurrently on this box ? expect slower wall-clock, and use -p no:cacheprovider exactly as given.
- CODE STYLE: match the repo ? rich docstrings that explain WHY and cite the design constraint (see claim_due_job's existing docstring for tone); comments only where the code can't say it; small functions. backend/main.py has an enforced line budget (tests/test_main_module_budget.py) ? avoid growing it.
- Your final message is consumed by the orchestrator via the structured schema. Report HONESTLY ? if a gate is red or you could not finish, say so in the schema fields; never claim green that you didn't see.

REVIEWER BLOCKER (verbatim, from PR #468):

"claim_due_job() is CAS-safe for avoiding double-claims, but under parallel drain workers the losers return None (backend/infrastructure/database_jobs.py:357-380). _drain_due_jobs() then treats None as 'queue empty' and exits (backend/api/job_worker.py:273-276). With max_workers > 1, multiple workers can select the same first queued row; one updates it, and the losers stop even though later jobs are still queued.

I reproduced locally on this head with 8 queued jobs and 8 concurrent claimers:

```
[None, None, None, None, 'j0', None, None, 'j1'] -> only 2/8 claims
```

For long Cloud Run bucket-poll jobs this can leave dispatch capacity unused and queued jobs waiting until another submit/wake or a running worker finally frees up, which undermines the

concurrency improvement this PR is trying to provide.

Suggested fix: make `claim_due_job()` retry on `rowcount != 1` until it either successfully claims a row or a fresh `SELECT` finds no runnable rows, or return a distinct lost-race sentinel so `_drain_due_jobs()` loops instead of treating it as empty. Please add a regression test with concurrent claimers/drains over `N` queued jobs proving there is no false-empty result / that `N` distinct jobs can be claimed."

TASK: Fix the false-empty lost-CAS-race bug in `claim_due_job()`.

- File: `backend/infrastructure/database_jobs.py` ? `claim_due_job()` (~line 357). Current shape: `SELECT one runnable row -> CAS UPDATE on (id, status) -> "if cur.rowcount != 1: return None # lost the race"`.
- Preferred fix (reviewer's first suggestion): retry `INSIDE` `claim_due_job()` ? loop: re-`SELECT`; if no runnable row, return `None` (now `None` truly means nothing-runnable); attempt the `CAS`; on `rowcount==1` return the claimed job; on a lost race, loop again. An unbounded loop is **CORRECT** and safe here: every lost race means another worker successfully claimed that row, so global progress is guaranteed (lock-free progress argument) ? do **NOT** add a retry cap that can reintroduce a false-empty. Keep the whole loop inside the existing `try/except` and keep the connection usage pattern consistent with the current code (decide deliberately whether to re-open the connection per attempt or reuse it ? look at `_connect()` and how the `sqlite` connection/transaction is configured, and pick what cannot deadlock or hold a write lock across iterations).
- Update the docstring: explain the lost-race retry and that `None` now means "nothing runnable", citing the parallel-drain design of this PR.
- `_drain_due_jobs()` in `backend/api/job_worker.py` should then need **NO** change (`None` = genuinely empty) ? verify this and say so.
- **REGRESSION TEST** (required by reviewer): add to `tests/test_job_worker_loop.py` (or the best-fitting existing job test file among `tests/test_job_self_scheduling.py` / `tests/test_job_worker_loop.py`): queue `N` jobs (`N>=8`), start `N` concurrent claimer threads synchronized with a `threading.Barrier` so they genuinely race, each calling `db.claim_due_job()` once; assert **ALL** `N` return non-`None` and the `N` claimed ids are **DISTINCT** (no false-empty, no double-claim). Study how existing tests build the jobs `DB/fixtures` and reuse that. The test must be deterministic and **CI-safe** on **Linux AND Windows** ? mind `SQLite` locking (check what `busy timeout _connect()` sets; if lost-race retries hit 'database is locked' under thread pressure, that is a real finding ? handle it the way the codebase already handles `sqlite` contention, and report what you did).
- Do **NOT** implement the park-vs-block (`JobWaiting/WAIT_AND_RESUME`) refinement ? it was offered to the reviewer separately and they have not asked for it.

When done, fill the structured output schema exactly and honestly.

2. user (2026-07-04T08:48:05.939Z)

fix/drain-concurrency

8534077 review: gate concurrency on effective compute path, not the server default (#466)

3. user (2026-07-04T08:48:10.875Z)

```
1     import json
2     import logging
3     from typing import Any, Dict, List, Optional, Tuple
4
5     from backend import billing # pure cost math (no backend imports -> no cycle)
6
7     logger = logging.getLogger(__name__)
8
9
10    class DatabaseJobsMixin:
11        def _connect(self): # pragma: no cover - implemented by Database facade
12            raise NotImplementedError
13
14        def _execute_write( # pragma: no cover - implemented by Database facade
15            self, sql: str, params: Tuple[Any, ...], error_msg: str
16        ) -> bool:
17            raise NotImplementedError
18
```

```

19 # --- Async job store (DEFERRED_HARDENING_PLAN #16) -----
20 # Writers follow the repo convention (swallow + logger.error, return bool); the
21 # worker logs loudly on a failed transition so a job can't silently wedge.
22
23 def create_job(
24     self,
25     job_id: str,
26     video_path: str,
27     segmenter: Optional[str] = None,
28     player_pool: Optional[List[str]] = None,
29     max_frames: Optional[int] = None,
30     policy: Optional[Dict[str, Any]] = None,
31     owner_id: Optional[str] = None,
32     locale: Optional[str] = None,
33     compute: Optional[str] = None,
34     force: bool = False,
35 ) -> bool:
36     """Insert a new job in state 'queued'. ``policy`` is an optional JSON
ExecutionPolicy
37     (EXECUTION_POLICY.md) the worker honours; None ? the worker's default policy.
``owner_id``
38     (M9, D9) is the verified tenant from edge auth ? NULL for the local single-user
(default).
39     ``locale`` (v7) is the UI language the submission came from ? NULL when
unrecorded.
40     ``compute`` (v8) is the SPA compute-picker choice ("local"/"cloud") the worker
reconstructs +
41     honours; NULL ? the server default (byte-identical to pre-picker behaviour).
``force`` opts
42     into deliberate reprocessing; false leaves params NULL for old-row
compatibility."""
43     params = json.dumps({"force": True}) if force else None
44     return self._execute_write(
45         "INSERT INTO jobs (id, video_path, segmenter, player_pool, max_frames,
status, policy, "
46         "owner_id, locale, compute, params) VALUES (?, ?, ?, ?, ?, 'queued', ?, ?,
?, ?, ?)",
47         (
48             job_id,
49             video_path,
50             segmenter,
51             json.dumps(player_pool) if player_pool is not None else None,
52             max_frames,
53             json.dumps(policy) if policy is not None else None,
54             owner_id,
55             locale,
56             compute,
57             params,
58         ),
59         f"Failed to create job {job_id}",
60     )
61
62 def create_compile_job(
63     self,
64     job_id: str,
65     video_id: str,
66     video_path: str,
67     segment_ids: List[int],
68     output_filename: str,
69     locale: Optional[str] = None,
70     owner_id: Optional[str] = None,
71 ) -> bool:
72     """Insert an async REEL-COMPILE job (#329) in state 'queued', `kind`='compile'.
The stitch
73     params (segment_ids / output_filename / locale) ride the JSON `params` column

```

```

since they don't
74         map onto the process columns; `video_path` is the target video's source ref (the
table's
75         NOT NULL column) and `video_id` is set up-front (so /api/jobs surfaces it
immediately). The
76         worker dispatches on `kind` and runs the ffmpeg stitch off the request
thread."""
77         params = json.dumps(
78             {
79                 "segment_ids": list(segment_ids),
80                 "output_filename": output_filename,
81                 "locale": locale,
82             }
83         )
84         return self._execute_write(
85             "INSERT INTO jobs (id, video_path, video_id, status, owner_id, locale, kind,
params) "
86             "VALUES (?, ?, ?, 'queued', ?, ?, 'compile', ?)",
87             (job_id, video_path, video_id, owner_id, locale, params),
88             f"Failed to create compile job {job_id}",
89         )
90
91     def count_jobs_by_locale(self) -> Dict[str, int]:
92         """PMF analytics: how many jobs were submitted per UI locale. Unrecorded locales
(NULL ?
93         CLI / pre-v7 / older clients) bucket under '(unset)'. Returns {locale: count},
busiest first."""
94         with self._connect() as conn:
95             rows = (
96                 conn.cursor()
97                 .execute(
98                     "SELECT COALESCE(locale, '(unset)') AS loc, COUNT(*) AS n "
99                     "FROM jobs GROUP BY loc ORDER BY n DESC, loc"
100                 )
101                 .fetchall()
102             )
103         return {str(r[0]): int(r[1]) for r in rows}
104
105     def mark_job_running(self, job_id: str) -> bool:
106         return self._execute_write(
107             "UPDATE jobs SET status='running', started_at=CURRENT_TIMESTAMP, "
108             "progress='starting' WHERE id = ?",
109             (job_id,),
110             f"Failed to mark job {job_id} running",
111         )
112
113     def set_job_progress(self, job_id: str, progress: str) -> bool:
114         return self._execute_write(
115             "UPDATE jobs SET progress=? WHERE id = ?",
116             (progress, job_id),
117             f"Failed to set progress for job {job_id}",
118         )
119
120     def set_job_video_id(self, job_id: str, video_id: str) -> bool:
121         return self._execute_write(
122             "UPDATE jobs SET video_id=? WHERE id = ?",
123             (video_id, job_id),
124             f"Failed to set video_id for job {job_id}",
125         )
126
127     def mark_job_done(self, job_id: str, result: Dict[str, Any]) -> bool:
128         """Job EXECUTED to completion. The pipeline's own verdict (success/failed
validation) is inside `result` ? job status 'done' means 'the worker
finished'."""
129
130         return self._execute_write(

```

```

131         "UPDATE jobs SET status='done', progress='finished', result=?, "
132         "finished_at=CURRENT_TIMESTAMP WHERE id = ?",
133         (json.dumps(result), job_id),
134         f"Failed to mark job {job_id} done",
135     )
136
137     def mark_job_failed(self, job_id: str, error: str) -> bool:
138         """Worker crashed / raised ? infrastructure failure, not a pipeline verdict."""
139         return self._execute_write(
140             "UPDATE jobs SET status='failed', error=?, "
141             "finished_at=CURRENT_TIMESTAMP WHERE id = ?",
142             (error, job_id),
143             f"Failed to mark job {job_id} failed",
144         )
145
146     def get_job(self, job_id: str) -> Optional[Dict[str, Any]]:
147         """Fetch one job; `result`/`player_pool` JSON deserialized."""
148         with self._connect() as conn:
149             row = (
150                 conn.cursor()
151                 .execute("SELECT * FROM jobs WHERE id = ?", (job_id,))
152                 .fetchone()
153             )
154             if not row:
155                 return None
156             d = dict(row)
157             d["result"] = json.loads(d["result"]) if d["result"] else None
158             d["player_pool"] = (
159                 json.loads(d["player_pool"]) if d["player_pool"] else None
160             )
161             d["policy"] = json.loads(d["policy"]) if d.get("policy") else None
162             return d
163
164     # --- Per-job cost ledger (COMPUTE_DECOUPLED_SERVING M8, D8)
165     -----
166     def get_pricebook_rates(
167         self, version: str, gpu_type: Optional[str] = None
168     ) -> Dict[str, float]:
169         """Resolve a flat ``{resource: unit_price_usd}`` for a pricebook ``version``.
170         The per-second GPU rate is selected by ``gpu_type``; non-GPU resources (wall/egress) carry
171         ``gpu_type=''``.
172         An unknown version ? ``{}`` (so every cost computes to 0.0)."""
173         gt = gpu_type or ""
174         rates: Dict[str, float] = {}
175         with self._connect() as conn:
176             rows = (
177                 conn.cursor()
178                 .execute(
179                     "SELECT resource, gpu_type, unit_price_usd FROM pricebook WHERE
180                     version = ?",
181                     (version,),
182                 )
183                 .fetchall()
184             )
185             for r in rows:
186                 res, row_gt, price = (
187                     r["resource"],
188                     (r["gpu_type"] or ""),
189                     float(r["unit_price_usd"]),
190                 )
191                 if res == billing.GPU_SECONDS:
192                     if row_gt == gt: # the GPU rate for THIS gpu_type
193                         rates[res] = price
194                 else:

```

```

192         rates[res] = price # wall/egress: gpu_type-independent
193     return rates
194
195     def record_job_cost(
196         self,
197         job_id: str,
198         *,
199         usage: Dict[str, float],
200         pricebook_version: str,
201         gpu_type: Optional[str] = None,
202         compute_backend: Optional[str] = None,
203         owner_id: Optional[str] = None,
204         margin: float = 0.0,
205     ) -> Optional[Dict[str, Any]]:
206         """Compute the job's cost from ``usage`` x the pricebook + persist every cost
column.
207         Returns the stored cost dict, or ``None`` on write failure. The caller gates
this on
208         ``billing.enabled`` (default-OFF) ? nothing records until billing is turned
on."""
209         rates = self.get_pricebook_rates(pricebook_version, gpu_type)
210         # SILENT-UNDER-BILL GUARD: a GPU-seconds AMOUNT with no RATE for this gpu_type
would bill at
211         # $0 indistinguishably from a free run (a new SKU / typo / gpu_type=None).
Surface it loudly +
212         # in the breakdown so the gap is auditable, not invisible. (A missing amount =
$0 IS safe.)
213         gpu_amt = float(usage.get(billing.GPU_SECONDS, 0.0) or 0.0)
214         unpriced_gpu = gpu_amt > 0 and billing.GPU_SECONDS not in rates
215         if unpriced_gpu:
216             logger.warning(
217                 "[BILLING] no pricebook rate for gpu_type=%r in version=%r ? %.1f GPU-
seconds billed "
218                 "at $0 (UNPRICED; recorded in cost_breakdown_json). Add a rate or fix
gpu_type.",
219                 gpu_type,
220                 pricebook_version,
221                 gpu_amt,
222             )
223         cost_usd, breakdown = billing.compute_cost_usd(usage, rates)
224         if unpriced_gpu:
225             breakdown["unpriced_gpu_seconds"] = gpu_amt
226         if margin:
227             marked = billing.apply_margin(cost_usd, margin)
228             breakdown["margin"] = round(
229                 marked - cost_usd, 6
230             ) # keep ?(breakdown) == cost_usd
231             cost_usd = marked
232         bytes_out = int(float(usage.get(billing.EGRESS_GB, 0.0) or 0.0) * 1e9)
233         ok = self._execute_write(
234             "UPDATE jobs SET owner_id=?, compute_backend=?, gpu_type=?,
gpu_active_seconds=?, "
235             "wall_seconds=?, bytes_out=?, gemini_cost_usd=?, pricebook_version=?,
cost_usd=?, "
236             "cost_breakdown_json=? WHERE id = ?",
237             (
238                 owner_id,
239                 compute_backend,
240                 gpu_type,
241                 usage.get(billing.GPU_SECONDS),
242                 usage.get(billing.WALL_SECONDS),
243                 bytes_out,
244                 usage.get(billing.GEMINI_USD),
245                 pricebook_version,
246                 cost_usd,

```

```

247         json.dumps(breakdown),
248         job_id,
249     ),
250     f"Failed to record cost for job {job_id}",
251 )
252 if not ok:
253     return None
254 return {
255     "cost_usd": cost_usd,
256     "cost_breakdown": breakdown,
257     "pricebook_version": pricebook_version,
258 }
259
260 def get_job_cost(self, job_id: str) -> Optional[Dict[str, Any]]:
261     """The job's cost row (``None`` if the job doesn't exist; ``cost_usd`` is
262     ``None`` until
263     recorded). ``cost_breakdown`` is the deserialized JSON breakdown."""
264     with self._connect() as conn:
265         row = (
266             conn.cursor()
267             .execute(
268                 "SELECT id, video_id, owner_id, compute_backend, gpu_type,
269                 gpu_active_seconds, "
270                 "wall_seconds, bytes_out, gemini_cost_usd, pricebook_version,
271                 cost_usd, "
272                 "cost_breakdown_json FROM jobs WHERE id = ?",
273                 (job_id,),
274             )
275             .fetchone()
276         )
277         if not row:
278             return None
279         d = dict(row)
280         raw = d.pop("cost_breakdown_json", None)
281         d["cost_breakdown"] = json.loads(raw) if raw else {}
282         return d
283
284 def get_video_cost(self, video_id: str) -> Dict[str, Any]:
285     """Aggregate cost across a video's jobs: total ``cost_usd`` + the per-job rows.
286     A video is
287     1:1 with an infer job, but a re-ingest can produce several ? sum them."""
288     with self._connect() as conn:
289         rows = (
290             conn.cursor()
291             .execute(
292                 "SELECT id, cost_usd, pricebook_version, cost_breakdown_json FROM
293                 jobs "
294                 "WHERE video_id = ?",
295                 (video_id,),
296             )
297             .fetchall()
298         )
299         jobs: List[Dict[str, Any]] = []
300         total = 0.0
301         for r in rows:
302             cost = r["cost_usd"]
303             if cost is not None:
304                 total += float(cost)
305             jobs.append(
306                 {
307                     "job_id": r["id"],
308                     "cost_usd": cost,
309                     "pricebook_version": r["pricebook_version"],
310                     "cost_breakdown": json.loads(r["cost_breakdown_json"])
311                     if r["cost_breakdown_json"]

```

```

307         else {},
308     }
309 )
310     return {"video_id": video_id, "total_cost_usd": round(total, 6), "jobs": jobs}
311
312     # --- Self-scheduling execution policy (EXECUTION_POLICY.md P2)
313     -----
314     def set_job_policy(self, job_id: str, policy: Optional[Dict[str, Any]]) -> bool:
315         """Attach/replace a job's JSON ExecutionPolicy."""
316         return self._execute_write(
317             "UPDATE jobs SET policy=? WHERE id = ?",
318             (json.dumps(policy) if policy is not None else None, job_id),
319             f"Failed to set policy for job {job_id}",
320         )
321
322     def set_job_waiting(
323         self, job_id: str, delay_sec: float, progress: Optional[str] = None
324     ) -> bool:
325         """Self-schedule: park a job in 'waiting' until ``delay_sec`` from now,
326         releasing the
327         worker. ``claim_due_job`` re-picks it when due ? cooperative, no master.
328         ``next_poll_at`` is
329         computed in SQLite's own UTC format so it compares directly to
330         CURRENT_TIMESTAMP."""
331         return self._execute_write(
332             "UPDATE jobs SET status='waiting', "
333             "next_poll_at=datetime('now', ?), "
334             "progress=COALESCE(?, progress) WHERE id = ?",
335             (f"+{max(0, int(delay_sec))} seconds", progress, job_id),
336             f"Failed to set job {job_id} waiting",
337         )
338
339     def seconds_until_next_due(self) -> Optional[float]:
340         """Seconds until the soonest parked ('waiting') job becomes due, so the worker
341         can
342         schedule a wake-up (EXECUTION_POLICY.md P3 self-scheduling). Returns 0.0 if one
343         is
344         already due, or None if nothing is waiting. A NULL ``next_poll_at`` ? due now
345         (0.0)."""
346         try:
347             with self._connect() as conn:
348                 row = (
349                     conn.cursor()
350                     .execute(
351                         "SELECT MIN(CASE WHEN next_poll_at IS NULL THEN 0 ELSE "
352                         " (julianday(next_poll_at) - julianday('now')) * 86400.0 END)
353                         AS secs "
354                         "FROM jobs WHERE status='waiting'"
355                     )
356                     .fetchone()
357                 )
358                 if row is None or row["secs"] is None:
359                     return None
360                 return max(0.0, float(row["secs"]))
361         except Exception as e:
362             logger.error(f"Failed to compute next-due delay: {e}")
363             return None
364
365     def claim_due_job(self) -> Optional[Dict[str, Any]]:
366         """Atomically claim ONE runnable job ? 'queued', or 'waiting' whose
367         ``next_poll_at`` is due ?
368         flipping it to 'running'. Compare-and-set on (id, status) so concurrent workers
369         never
370         double-claim (the DB row-claim is the only coordination needed; no master).
371         Earliest-due

```

```

361         first. Returns the claimed job dict, or None if nothing is runnable."""
362     try:
363         with self._connect() as conn:
364             cur = conn.cursor()
365             row = cur.execute(
366                 "SELECT id, status FROM jobs WHERE status='queued' "
367                 "OR (status='waiting' AND (next_poll_at IS NULL OR next_poll_at <=
CURRENT_TIMESTAMP)) "
368                 "ORDER BY COALESCE(next_poll_at, created_at) ASC LIMIT 1"
369             ).fetchone()
370             if not row:
371                 return None
372             cur.execute(
373                 "UPDATE jobs SET status='running', started_at=CURRENT_TIMESTAMP "
374                 "WHERE id=? AND status=?",
375                 (row["id"], row["status"]),
376             )
377             conn.commit()
378             if cur.rowcount != 1:
379                 return None # lost the race to a concurrent worker
380             return self.get_job(row["id"])
381     except Exception as e:
382         logger.error(f"Failed to claim a due job: {e}")
383         return None
384
385     def fail_stale_jobs(self, reason: str = "interrupted by server restart") -> int:
386         """Crash recovery: any job still queued/running from a previous process is dead
387         (the executor is in-process). Mark failed so clients see a terminal state, not a
388         hang. Returns the number of jobs swept.
389
390         NOTE: ``recover_stale_jobs`` is the preferred startup call ? it RE-QUEUES stale
391         ``compile`` jobs (idempotent CPU stitch, safe to retry) instead of failing them.
392         This
393         blanket-fail method stays for callers/tests that want the old behaviour."""
394     try:
395         with self._connect() as conn:
396             cur = conn.cursor()
397             cur.execute(
398                 "UPDATE jobs SET status='failed', error=?,
finished_at=CURRENT_TIMESTAMP "
399                 "WHERE status IN ('queued', 'running')",
400                 (reason,),
401             )
402             conn.commit()
403             return cur.rowcount
404     except Exception as e:
405         logger.error(f"Failed to sweep stale jobs: {e}")
406         return 0
407
408     def recover_stale_jobs(
409         self, reason: str = "interrupted by server restart"
410     ) -> Tuple[int, int]:
411         """Crash recovery, kind-aware (#329 serving reliability). Called once at
startup.
412
413         A ``compile`` job is a **CPU-only, idempotent** ffmpeg re-stitch (re-resolving
the same
414         segments yields the same reel), so a compile left queued/running by a killed
instance is
415         **re-queued** ? the startup drain re-runs it and the client keeps polling until
it gets
416         the reel, instead of seeing a spurious "failed". A ``process`` job is an
expensive,
417         non-idempotent GPU detection pass; auto-retrying could double-spend, so it is
**failed**

```

```

417         (terminal, as before) and the client re-submits deliberately.
418
419     Returns `` (requeued_compile, failed_process) ``. """
420     try:
421         with self._connect() as conn:
422             cur = conn.cursor()
423             # Re-queue interrupted COMPILE jobs (idempotent ? safe to retry
automatically).
424             cur.execute(
425                 "UPDATE jobs SET status='queued', started_at=NULL,
next_poll_at=NULL, "
426                 "error=NULL WHERE status IN ('queued', 'running') AND
kind='compile'"
427             )
428             requeued = cur.rowcount
429             # Fail interrupted PROCESS jobs (GPU, not safe to auto-retry).
430             cur.execute(
431                 "UPDATE jobs SET status='failed', error=?,
finished_at=CURRENT_TIMESTAMP "
432                 "WHERE status IN ('queued', 'running') AND (kind IS NULL OR kind !=
'compile')",
433                 (reason, ),
434             )
435             failed = cur.rowcount
436             conn.commit()
437             return requeued, failed
438     except Exception as e:
439         logger.error(f"Failed to recover stale jobs: {e}")
440         return 0, 0
441
442

```

4. user (2026-07-04T08:48:11.815Z)

```

230         billing_cfg.pricebook_version,
231     )
232     except Exception as e: # never wedge a completed job on bookkeeping
233         logger.warning(
234             "Cost recording for job %s failed (non-fatal): %s", job.get("id"), e
235         )
236
237
238     def _maybe_run_flywheel(
239         job: Dict[str, Any], result: Dict[str, Any], config: Any
240     ) -> None:
241         """M11 data-flywheel (COMPUTE_DECOUPLED_SERVING M11, D12). DEFAULT-OFF + INERT: a
no-op unless
242         ``config.flywheel.enabled`` AND attested consent ? and consent is a **faked hard-
deny** today
243         (``flywheel.consent_attested`` returns False for every job until counsel + the
consent schema
244         land), so this captures **nothing** in production. When activated it captures the
consented job's
245         artifacts into the per-video workspace (? later labeling ? the golden training set).
246
247         Best-effort ? never fails a completed job because the flywheel hiccuped. ``job`` is
the claimed
248         row (carries ``owner_id``); ``result`` is the pipeline output. """
249
250         flywheel_cfg = getattr(config, "flywheel", None)
251         if not flywheel_cfg or not getattr(flywheel_cfg, "enabled", False):
252             return
253         try:
254             from backend import flywheel
255

```

```

256         flywheel.run_for_job(job, result, config)
257     except Exception as e: # never wedge a completed job on the flywheel
258         logger.warning("Flywheel for job %s failed (non-fatal): %s", job.get("id"), e)
259
260
261     def _drain_due_jobs(config: Any, db: Any) -> int:
262         """One worker tick: claim and run every currently-runnable job (queued, or waiting
whose
263         `next_poll_at` is due), earliest-due first, until the table has nothing runnable.
264
265         Each parked job (`JobWaiting` ? status 'waiting') drops out of this drain and is
picked
266         up by a FUTURE drain once due; we schedule that follow-up drain via
`_schedule_drain` so
267         a single-worker box keeps making progress with no central timer (the DB is the
clock).
268         Returns the number of jobs that reached a terminal/parked state this tick.
269         """
270         import backend.main as _main
271
272         ran = 0
273         while True:
274             job = db.claim_due_job()
275             if job is None:
276                 break
277             ran += 1
278             _main._run_claimed_job(job, config, db)
279             # If anything is parked-but-not-yet-due, arrange a wake-up so it resumes without a
new
280             # submit. On the single-box executor this is a delayed re-drain; multi-worker just
relies
281             # on the next submit/drain. Best-effort ? never raises into the worker.
282             _main._schedule_next_drain_if_waiting(config, db)
283             return ran
284
285
286     def _schedule_next_drain_if_waiting(config: Any, db: Any) -> None:
287         """If any job is parked in 'waiting', arm a re-drain for when the soonest one is
due, so a
288         self-scheduled job resumes with no further client submit. The drain itself re-checks
289         due-ness, so an early wake is harmless and a clamped wake for a far-future job is
fine."""
290         import backend.main as _main
291
292         try:
293             delay = db.seconds_until_next_due()
294         except Exception as e: # pragma: no cover - defensive; never wedge the worker
295             logger.error(f"Could not compute next-due delay: {e}")
296             return
297         if delay is None:
298             return # nothing waiting
299         # Clamp so a far-future job doesn't pin a thread forever; the drain re-checks due-
ness.
300         _main._arm_wake_timer(max(0.0, min(delay, _main._MAX_DRAIN_WAKE_SEC)), config, db)
301
302
303     def _arm_wake_timer(delay_sec: float, config: Any, db: Any) -> None:
304         """Arm a single daemon timer that re-submits a drain after ``delay_sec`` (no central
305         scheduler ? the DB is the clock). Isolated as the sole timer-creating seam so tests
can
306         monkeypatch it to a no-op and assert the *scheduling decision* without real
waiting."""
307         import backend.main as _main
308
309         timer = threading.Timer(

```

5. user (2026-07-04T08:48:17.932Z)

```
C:\Users\avidu\Projects\kheIsutra-guru\wt-drain-
concurrency\backend\infrastructure\database_jobs.py-6-
C:\Users\avidu\Projects\kheIsutra-guru\wt-drain-
concurrency\backend\infrastructure\database_jobs.py-7-logger = logging.getLogger(__name__)
C:\Users\avidu\Projects\kheIsutra-guru\wt-drain-
concurrency\backend\infrastructure\database_jobs.py-8-
C:\Users\avidu\Projects\kheIsutra-guru\wt-drain-
concurrency\backend\infrastructure\database_jobs.py-9-
C:\Users\avidu\Projects\kheIsutra-guru\wt-drain-
concurrency\backend\infrastructure\database_jobs.py-10-class DatabaseJobsMixin:
C:\Users\avidu\Projects\kheIsutra-guru\wt-drain-
concurrency\backend\infrastructure\database_jobs.py:11:     def _connect(self): # pragma: no
cover - implemented by Database facade
C:\Users\avidu\Projects\kheIsutra-guru\wt-drain-
concurrency\backend\infrastructure\database_jobs.py-12-         raise NotImplementedError
C:\Users\avidu\Projects\kheIsutra-guru\wt-drain-
concurrency\backend\infrastructure\database_jobs.py-13-
C:\Users\avidu\Projects\kheIsutra-guru\wt-drain-
concurrency\backend\infrastructure\database_jobs.py-14-     def _execute_write( # pragma: no
cover - implemented by Database facade
C:\Users\avidu\Projects\kheIsutra-guru\wt-drain-
concurrency\backend\infrastructure\database_jobs.py-15-         self, sql: str, params:
Tuple[Any, ...], error_msg: str
C:\Users\avidu\Projects\kheIsutra-guru\wt-drain-
concurrency\backend\infrastructure\database_jobs.py-16-     ) -> bool:
--
C:\Users\avidu\Projects\kheIsutra-guru\wt-drain-
concurrency\backend\infrastructure\database_user_models.py-4-
C:\Users\avidu\Projects\kheIsutra-guru\wt-drain-
concurrency\backend\infrastructure\database_user_models.py-5-logger =
logging.getLogger(__name__)
C:\Users\avidu\Projects\kheIsutra-guru\wt-drain-
concurrency\backend\infrastructure\database_user_models.py-6-
C:\Users\avidu\Projects\kheIsutra-guru\wt-drain-
concurrency\backend\infrastructure\database_user_models.py-7-
C:\Users\avidu\Projects\kheIsutra-guru\wt-drain-
concurrency\backend\infrastructure\database_user_models.py-8-class DatabaseUserModelsMixin:
C:\Users\avidu\Projects\kheIsutra-guru\wt-drain-
concurrency\backend\infrastructure\database_user_models.py:9:     def _connect(self): # pragma:
no cover - implemented by Database facade
C:\Users\avidu\Projects\kheIsutra-guru\wt-drain-
concurrency\backend\infrastructure\database_user_models.py-10-         raise NotImplementedError
C:\Users\avidu\Projects\kheIsutra-guru\wt-drain-
concurrency\backend\infrastructure\database_user_models.py-11-
C:\Users\avidu\Projects\kheIsutra-guru\wt-drain-
concurrency\backend\infrastructure\database_user_models.py-12-     def _execute_write( # pragma:
no cover - implemented by Database facade
C:\Users\avidu\Projects\kheIsutra-guru\wt-drain-
concurrency\backend\infrastructure\database_user_models.py-13-         self, sql: str, params:
Tuple[Any, ...], error_msg: str
C:\Users\avidu\Projects\kheIsutra-guru\wt-drain-
concurrency\backend\infrastructure\database_user_models.py-14-     ) -> bool:
--
C:\Users\avidu\Projects\kheIsutra-guru\wt-drain-
concurrency\backend\infrastructure\database_media.py-4-
C:\Users\avidu\Projects\kheIsutra-guru\wt-drain-
concurrency\backend\infrastructure\database_media.py-5-logger = logging.getLogger(__name__)
C:\Users\avidu\Projects\kheIsutra-guru\wt-drain-
concurrency\backend\infrastructure\database_media.py-6-
C:\Users\avidu\Projects\kheIsutra-guru\wt-drain-
concurrency\backend\infrastructure\database_media.py-7-
C:\Users\avidu\Projects\kheIsutra-guru\wt-drain-
concurrency\backend\infrastructure\database_media.py-8-class DatabaseMediaMixin:
```

```

C:\Users\avidu\Projects\khealsutra-guru\wt-drain-
concurrency\backend\infrastructure\database_media.py:9:   def _connect(self): # pragma: no
cover - implemented by Database facade
C:\Users\avidu\Projects\khealsutra-guru\wt-drain-
concurrency\backend\infrastructure\database_media.py:10-       raise NotImplementedError
C:\Users\avidu\Projects\khealsutra-guru\wt-drain-
concurrency\backend\infrastructure\database_media.py:11-
C:\Users\avidu\Projects\khealsutra-guru\wt-drain-
concurrency\backend\infrastructure\database_media.py:12-   def _execute_write( # pragma: no
cover - implemented by Database facade
C:\Users\avidu\Projects\khealsutra-guru\wt-drain-
concurrency\backend\infrastructure\database_media.py:13-       self, sql: str, params:
Tuple[Any, ...], error_msg: str
C:\Users\avidu\Projects\khealsutra-guru\wt-drain-
concurrency\backend\infrastructure\database_media.py:14-   ) -> bool:
--
C:\Users\avidu\Projects\khealsutra-guru\wt-drain-
concurrency\backend\infrastructure\database.py:27-   def __init__(self, db_path: str):
C:\Users\avidu\Projects\khealsutra-guru\wt-drain-
concurrency\backend\infrastructure\database.py:28-       self.db_path = db_path
C:\Users\avidu\Projects\khealsutra-guru\wt-drain-
concurrency\backend\infrastructure\database.py:29-       self._init_db()
C:\Users\avidu\Projects\khealsutra-guru\wt-drain-
concurrency\backend\infrastructure\database.py:30-
C:\Users\avidu\Projects\khealsutra-guru\wt-drain-
concurrency\backend\infrastructure\database.py:31-   @contextmanager
C:\Users\avidu\Projects\khealsutra-guru\wt-drain-
concurrency\backend\infrastructure\database.py:32-   def _connect(self):
C:\Users\avidu\Projects\khealsutra-guru\wt-drain-
concurrency\backend\infrastructure\database.py:33-       """Transaction scope that also CLOSES
the connection.
C:\Users\avidu\Projects\khealsutra-guru\wt-drain-
concurrency\backend\infrastructure\database.py:34-
C:\Users\avidu\Projects\khealsutra-guru\wt-drain-
concurrency\backend\infrastructure\database.py:35-       ``with sqlite3.connect(...)`` alone is
only a commit/rollback scope ? it
C:\Users\avidu\Projects\khealsutra-guru\wt-drain-
concurrency\backend\infrastructure\database.py:36-       never closes, so every call leaked its
connection to GC (lingering file
C:\Users\avidu\Projects\khealsutra-guru\wt-drain-
concurrency\backend\infrastructure\database.py:37-       handles on the WAL sidecars under
Windows). Internal methods use this.
--
C:\Users\avidu\Projects\khealsutra-guru\wt-drain-
concurrency\backend\infrastructure\database.py:42-       yield conn
C:\Users\avidu\Projects\khealsutra-guru\wt-drain-
concurrency\backend\infrastructure\database.py:43-       finally:
C:\Users\avidu\Projects\khealsutra-guru\wt-drain-
concurrency\backend\infrastructure\database.py:44-       conn.close()
C:\Users\avidu\Projects\khealsutra-guru\wt-drain-
concurrency\backend\infrastructure\database.py:45-
C:\Users\avidu\Projects\khealsutra-guru\wt-drain-
concurrency\backend\infrastructure\database.py:46-   def _get_connection(self):
C:\Users\avidu\Projects\khealsutra-guru\wt-drain-
concurrency\backend\infrastructure\database.py:47-       # busy_timeout: wait (not fail) on a
locked DB ? needed once the async job model
C:\Users\avidu\Projects\khealsutra-guru\wt-drain-
concurrency\backend\infrastructure\database.py:48-       # adds concurrent writers (a swallowed
'database is locked' silently drops segments).
C:\Users\avidu\Projects\khealsutra-guru\wt-drain-
concurrency\backend\infrastructure\database.py:49-       conn = sqlite3.connect(self.db_path,
timeout=30.0)
C:\Users\avidu\Projects\khealsutra-guru\wt-drain-
concurrency\backend\infrastructure\database.py:50-       conn.row_factory = sqlite3.Row
C:\Users\avidu\Projects\khealsutra-guru\wt-drain-

```

```

concurrency\backend\infrastructure\database.py-51-      # Enforce ON DELETE CASCADE / SET NULL
? SQLite leaves FKs off per-connection.
C:\Users\avidu\Projects\khehsutra-guru\wt-drain-
concurrency\backend\infrastructure\database.py-52-      conn.execute("PRAGMA foreign_keys =
ON")
C:\Users\avidu\Projects\khehsutra-guru\wt-drain-
concurrency\backend\infrastructure\database.py:53-      # WAL allows concurrent readers during
a write; busy_timeout bounds lock waits.
C:\Users\avidu\Projects\khehsutra-guru\wt-drain-
concurrency\backend\infrastructure\database.py:54-      conn.execute("PRAGMA busy_timeout =
30000")
C:\Users\avidu\Projects\khehsutra-guru\wt-drain-
concurrency\backend\infrastructure\database.py-55-      try:
C:\Users\avidu\Projects\khehsutra-guru\wt-drain-
concurrency\backend\infrastructure\database.py-56-          conn.execute("PRAGMA journal_mode
= WAL")
C:\Users\avidu\Projects\khehsutra-guru\wt-drain-
concurrency\backend\infrastructure\database.py-57-      except sqlite3.OperationalError:
C:\Users\avidu\Projects\khehsutra-guru\wt-drain-
concurrency\backend\infrastructure\database.py-58-          pass # e.g. a read-only/networked
FS that can't do WAL ? degrade gracefully
C:\Users\avidu\Projects\khehsutra-guru\wt-drain-
concurrency\backend\infrastructure\database.py-59-      return conn
--
C:\Users\avidu\Projects\khehsutra-guru\wt-drain-
concurrency\backend\infrastructure\execution_policy.py-111-          _box["exc"] = e
C:\Users\avidu\Projects\khehsutra-guru\wt-drain-
concurrency\backend\infrastructure\execution_policy.py-112-      finally:
C:\Users\avidu\Projects\khehsutra-guru\wt-drain-
concurrency\backend\infrastructure\execution_policy.py-113-          _done.set()
C:\Users\avidu\Projects\khehsutra-guru\wt-drain-
concurrency\backend\infrastructure\execution_policy.py-114-
C:\Users\avidu\Projects\khehsutra-guru\wt-drain-
concurrency\backend\infrastructure\execution_policy.py-115-      threading.Thread(target=_runner, name=f"bounded:{label}", daemon=True).start()
C:\Users\avidu\Projects\khehsutra-guru\wt-drain-
concurrency\backend\infrastructure\execution_policy.py:116-      if
done.wait(timeout=policy.op_timeout_sec):
C:\Users\avidu\Projects\khehsutra-guru\wt-drain-
concurrency\backend\infrastructure\execution_policy.py-117-          if "exc" in box:
C:\Users\avidu\Projects\khehsutra-guru\wt-drain-
concurrency\backend\infrastructure\execution_policy.py-118-              last_exc = box["exc"]
C:\Users\avidu\Projects\khehsutra-guru\wt-drain-
concurrency\backend\infrastructure\execution_policy.py-119-              log.info(
C:\Users\avidu\Projects\khehsutra-guru\wt-drain-
concurrency\backend\infrastructure\execution_policy.py-120-                  "%s: attempt
%d/%d failed: %s",
C:\Users\avidu\Projects\khehsutra-guru\wt-drain-
concurrency\backend\infrastructure\execution_policy.py-121-                  label,

```

6. assistant (2026-07-04T08:48:45.004Z)

`\_connect()` yields a fresh connection per call (30s busy_timeout, WAL) and closes it on exit. Now let me look at the existing tests to reuse fixtures.

7. user (2026-07-04T08:48:45.480Z)

```

1      """P3 worker self-scheduling loop (EXECUTION_POLICY.md P3).
2
3      The P2 slice added the DB primitives (`claim_due_job`, `set_job_waiting`,
4      `seconds_until_next_due`); this slice wires the `main.py` worker to them so the worker
5      DRAINS the jobs table via `claim_due_job` and honours WAIT_AND_RESUME: a job that raises
6      `JobWaiting` is parked (status 'waiting' + `next_poll_at` + progress) and re-claimed
when
7      due ? instead of the old one-shot `submit(_run_job, ...)`.
```

```

8
9 Covered:
10 - `_drain_due_jobs` claims + runs queued jobs to a terminal state (no regression on the
11   existing sync/submit path ? same `done`/`failed`/verdict semantics);
12 - earliest-due ordering across queued + due-waiting;
13 - a `JobWaiting` parks the job (no terminal state) and a later drain (once due) resumes
it;
14 - not-yet-due parked jobs are left alone and a wake-timer is armed for them;
15 - the wake-scheduling decision (`_schedule_next_drain_if_waiting`) is asserted with the
16   timer seam stubbed ? no real waiting, no leaked threads.
17
18 The real-timer seam (`_arm_wake_timer`) is monkeypatched to a recorder in every test
that
19 can park a job, so nothing actually sleeps and no daemon timer outlives the test.
20 ""
21
22 import sqlite3
23
24 import pytest
25
26 pytest.importorskip("httpx")
27 from fastapi.testclient import TestClient
28
29 import backend.main as m
30 from backend.config.models import AppConfig
31 from backend.infrastructure.database import Database
32 from backend.pipeline.validators.base import ValidationResult
33
34
35 class _InlineExecutor:
36     """submit() runs fn synchronously ? deterministic, single-threaded tests."""
37
38     def submit(self, fn, *args, **kwargs):
39         fn(*args, **kwargs)
40
41
42 @pytest.fixture
43 def env(tmp_path, monkeypatch):
44     db = Database(str(tmp_path / "t.db"))
45     cfg = AppConfig()
46     monkeypatch.setattr(m, "_get_executor", lambda: _InlineExecutor())
47     # Stub the only real-timer seam so a parked job never arms a live daemon timer;
tests
48     # that care about scheduling assert against this recorder instead.
49     armed: list = []
50     monkeypatch.setattr(m, "_arm_wake_timer", lambda delay, *a: armed.append(delay))
51     app = m.create_app(cfg, db)
52     client = TestClient(app)
53     return client, db, tmp_path, monkeypatch, armed
54
55
56 def _video(tmp_path, name="m.mp4"):
57     p = tmp_path / name
58     p.write_bytes(b"not-a-real-video-but-hashable")
59     return p
60
61
62 def _pass():
63     return [ValidationResult(validator_name="x", passed=True, message="ok", details={})]
64
65
66 class _FakeSeg:
67     def __init__(self, db, cfg):
68         self.db = db
69

```

```

70     def process_video(self, video_id, video_path, *a, **k):
71         sid = self.db.add_play_segment(video_id, 0.0, 5.0)
72         return [
73             {
74                 "id": sid,
75                 "start_time": 0.0,
76                 "end_time": 5.0,
77                 "duration": 5.0,
78                 "metadata": {},
79             }
80         ]
81
82
83     def _wire_happy(mp, client_env_video):
84         mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
85         mp.setattr(m.segmenter_registry, "get", lambda name: _FakeSeg)
86
87
88     # --- no-regression: the drain runs a queued job exactly like the old submit path
89     -----
90
91     def test_drain_runs_queued_job_to_done(env):
92         client, db, tmp_path, mp, _armed = env
93         vid_file = _video(tmp_path)
94         _wire_happy(mp, vid_file)
95
96         # Submit via the public endpoint (which now submits a drain tick, not _run_job).
97         job_id = client.post(
98             "/api/process", json={"video_path": str(vid_file), "segmenter": "x"}
99         ).json()["job_id"]
100         body = client.get(f"/api/jobs/{job_id}").json()
101         assert body["status"] == "done"
102         assert body["result"]["status"] == "success"
103         assert len(body["result"]["play_segments"]) == 1
104         vid = m.compute_video_id(str(vid_file))
105         assert body["video_id"] == vid
106         assert len(db.query_play_segments(vid, {})) == 1
107
108
109     def test_drain_returns_count_and_drains_all_queued(env):
110         client, db, tmp_path, mp, _armed = env
111         _wire_happy(mp, None)
112         # Two queued rows created directly (no endpoint), distinct video_paths.
113         a, b = _video(tmp_path, "a.mp4"), _video(tmp_path, "b.mp4")
114         db.create_job("a", str(a), segmenter="x")
115         db.create_job("b", str(b), segmenter="x")
116
117         ran = m._drain_due_jobs(m.global_config, m.db_instance)
118         assert ran == 2
119         assert db.get_job("a")["status"] == "done"
120         assert db.get_job("b")["status"] == "done"
121         assert (
122             m._drain_due_jobs(m.global_config, m.db_instance) == 0
123         ) # nothing runnable left
124
125
126     def test_drain_empty_table_is_noop(env):
127         _client, _db, _tmp, _mp, _armed = env
128         assert m._drain_due_jobs(m.global_config, m.db_instance) == 0
129
130
131     # --- WAIT_AND_RESUME: a JobWaiting parks the job, a later drain resumes it
132     -----

```

```

133
134 def test_jobwaiting_parks_job_then_resumes_when_due(env):
135     client, db, tmp_path, mp, armed = env
136     vid_file = _video(tmp_path)
137     mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
138
139     # First execution parks the job (a FUTURE due time so it does NOT resume within the
same
140     # drain tick); the resume completes it. A one-shot toggle models "wait, then finish
next
141     # time" without a real external op.
142     state = {"parked": False}
143
144     class _WaitOnceSeg:
145         def __init__(self, db, cfg):
146             self.db = db
147
148         def process_video(self, video_id, video_path, *a, **k):
149             if not state["parked"]:
150                 state["parked"] = True
151                 raise m.JobWaiting(delay_sec=3600, progress="waiting on gemini")
152             sid = self.db.add_play_segment(video_id, 0.0, 5.0)
153             return [
154                 {
155                     "id": sid,
156                     "start_time": 0.0,
157                     "end_time": 5.0,
158                     "duration": 5.0,
159                     "metadata": {},
160                 }
161             ]
162
163     mp.setattr(m.segmenter_registry, "get", lambda name: _WaitOnceSeg)
164
165     job_id = client.post(
166         "/api/process", json={"video_path": str(vid_file), "segmenter": "x"}
167     ).json()["job_id"]
168
169     # After the first drain the job is PARKED, not terminal, and its progress persisted.
170     parked = db.get_job(job_id)
171     assert parked["status"] == "waiting"
172     assert parked["progress"] == "waiting on gemini"
173     assert parked["next_poll_at"] is not None
174     assert parked["result"] is None # not done, not failed
175     # A wake-timer was armed for the parked job (clamped to the cap).
176     assert armed and 0.0 < armed[-1] <= m._MAX_DRAIN_WAKE_SEC
177
178     # Simulate the due time arriving: re-park as due-now, then drain re-claims and
finishes.
179     db.set_job_waiting(job_id, delay_sec=0)
180     m._drain_due_jobs(m.global_config, m.db_instance)
181     done = db.get_job(job_id)
182     assert done["status"] == "done"
183     assert done["result"]["status"] == "success"
184
185
186 def test_not_yet_due_waiting_job_is_left_and_wake_armed(env):
187     _client, db, _tmp, _mp, armed = env
188     db.create_job("j1", "/v.mp4")
189     db.set_job_waiting("j1", delay_sec=3600, progress="parked")
190
191     ran = m._drain_due_jobs(m.global_config, m.db_instance)
192     assert ran == 0 # not due -> not claimed
193     assert db.get_job("j1")["status"] == "waiting" # left parked
194     # A wake was scheduled for ~the due time (clamped to the cap).

```

```

195         assert armed and 0.0 < armed[-1] <= m._MAX_DRAIN_WAKE_SEC
196
197
198     def test_no_wake_armed_when_nothing_waiting(env):
199         client, db, tmp_path, mp, armed = env
200         vid_file = _video(tmp_path)
201         _wire_happy(mp, vid_file)
202         client.post("/api/process", json={"video_path": str(vid_file), "segmenter": "x"})
203         assert armed == [] # happy-path job terminalises; nothing parked -> no timer
204
205
206     # --- ordering: earliest-due first across queued + due-waiting
207     -----
208
209     def test_claim_orders_earliest_due_first(env):
210         _client, db, _tmp, _mp, _armed = env
211         # b is parked due-immediately; a is plain queued. Earliest COALESCE(next_poll_at,
212         # created_at) wins. Set b due in the past so it precedes a's created_at.
213         db.create_job("a", "/a.mp4")
214         db.create_job("b", "/b.mp4")
215         db.set_job_waiting("b", delay_sec=0)
216         order = []
217         while True:
218             job = db.claim_due_job()
219             if job is None:
220                 break
221             order.append(job["id"])
222         assert set(order) == {"a", "b"}
223
224
225     # --- seconds_until_next_due (the wake-delay computation)
226     -----
227
228     def test_seconds_until_next_due_none_when_no_waiting(tmp_path):
229         db = Database(str(tmp_path / "d.db"))
230         db.create_job("q", "/v.mp4") # queued, not waiting
231         assert db.seconds_until_next_due() is None
232
233
234     def test_seconds_until_next_due_zero_when_due(tmp_path):
235         db = Database(str(tmp_path / "d.db"))
236         db.create_job("j", "/v.mp4")
237         db.set_job_waiting("j", delay_sec=0)
238         assert db.seconds_until_next_due() == 0.0
239
240
241     def test_seconds_until_next_due_positive_for_future(tmp_path):
242         db = Database(str(tmp_path / "d.db"))
243         db.create_job("j", "/v.mp4")
244         db.set_job_waiting("j", delay_sec=120)
245         secs = db.seconds_until_next_due()
246         assert secs is not None and 60.0 < secs <= 120.0 # ~120s out, allowing clock slack
247
248
249     def test_seconds_until_next_due_picks_soonest(tmp_path):
250         db = Database(str(tmp_path / "d.db"))
251         db.create_job("far", "/a.mp4")
252         db.create_job("near", "/b.mp4")
253         db.set_job_waiting("far", delay_sec=3600)
254         db.set_job_waiting("near", delay_sec=30)
255         secs = db.seconds_until_next_due()
256         assert secs is not None and secs <= 30.0 # the nearer job dominates
257

```

```

258
259     # --- a crash during the drain is a normal failure (not a wedge)
-----
260
261
262     def test_drain_segementer_crash_marks_failed(env):
263         _client, db, tmp_path, mp, _armed = env
264         vid_file = _video(tmp_path)
265         mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
266
267         class _BoomSeg:
268             def __init__(self, db, cfg):
269                 self.db = db
270
271             def process_video(self, *a, **k):
272                 raise RuntimeError("kaboom")
273
274         mp.setattr(m.segementer_registry, "get", lambda name: _BoomSeg)
275         db.create_job("j", str(vid_file), segementer="x")
276         ran = m._drain_due_jobs(m.global_config, m.db_instance)
277         assert ran == 1
278         assert db.get_job("j")["status"] == "failed"
279         assert "RuntimeError" in db.get_job("j")["error"]
280
281
282     # --- migration sanity (the columns the loop depends on exist)
-----
283
284
285     def test_waiting_columns_present(tmp_path):
286         db = Database(str(tmp_path / "fresh.db"))
287         with db._get_connection() as conn:
288             cols = {r["name"] for r in conn.execute("PRAGMA table_info(jobs)").fetchall()}
289         assert {"policy", "next_poll_at", "status", "progress"} <= cols
290         # status='waiting' is a value, not a column; assert the round-trip works.
291         db.create_job("j", "/v.mp4")
292         db.set_job_waiting("j", delay_sec=5)
293         assert db.get_job("j")["status"] == "waiting"
294
295
296     def test_fresh_db_is_v9(tmp_path):
297         db = Database(str(tmp_path / "v.db"))
298         with db._get_connection() as conn:
299             # Head of the migration ladder (bump when database.py adds a new `if version <
300             N` ).
301             assert conn.execute("PRAGMA user_version").fetchone()[0] == 9
302
303     def test_legacy_db_upgrades_to_head(tmp_path):
304         # A pre-migration legacy DB (only `videos`, user_version 0) must run the FULL
305         ordered
306         # migration ladder up to the current head (v9) without error ? this exercises the
307         v0?v9
308         # path including the v1 `candidate_segments` CREATE that the later v4 ALTER depends
309         on.
310         path = str(tmp_path / "legacy.db")
311         conn = sqlite3.connect(path)
312         conn.execute(
313             "CREATE TABLE videos (id TEXT PRIMARY KEY, filepath TEXT NOT NULL, "
314             "processed_at TIMESTAMP, status TEXT NOT NULL, validation_results TEXT)"
315         )
316         conn.execute("PRAGMA user_version = 0")
317         conn.commit()
318         conn.close()
319         db = Database(path)

```

```

317         with db._get_connection() as c:
318             assert c.execute("PRAGMA user_version").fetchone()[0] == 9
319

```

8. assistant (2026-07-04T08:48:52.837Z)

Let me view the `\_connect` context manager fully and the PR's diff for context.

9. user (2026-07-04T08:48:54.569Z)

```

20
21
22     class SchemaTooNewError(RuntimeError):
23         """The on-disk DB was created by a newer app version than this binary understands
(P2d)."""
24
25
26     class Database(DatabaseMediaMixin, DatabaseJobsMixin, DatabaseUserModelsMixin):
27         def __init__(self, db_path: str):
28             self.db_path = db_path
29             self._init_db()
30
31         @contextmanager
32         def _connect(self):
33             """Transaction scope that also CLOSES the connection.
34
35             ``with sqlite3.connect(...)`` alone is only a commit/rollback scope ? it
36             never closes, so every call leaked its connection to GC (lingering file
37             handles on the WAL sidecars under Windows). Internal methods use this.
38             """
39             conn = self._get_connection()
40             try:
41                 with conn:
42                     yield conn
43             finally:
44                 conn.close()
45
46         def _get_connection(self):
47             # busy_timeout: wait (not fail) on a locked DB ? needed once the async job model
48             # adds concurrent writers (a swallowed 'database is locked' silently drops
segments).
49             conn = sqlite3.connect(self.db_path, timeout=30.0)
50             conn.row_factory = sqlite3.Row
51             # Enforce ON DELETE CASCADE / SET NULL ? SQLite leaves FKs off per-connection.
52             conn.execute("PRAGMA foreign_keys = ON")
53             # WAL allows concurrent readers during a write; busy_timeout bounds lock waits.
54             conn.execute("PRAGMA busy_timeout = 30000")
55             try:
56                 conn.execute("PRAGMA journal_mode = WAL")
57             except sqlite3.OperationalError:
58                 pass # e.g. a read-only/networked FS that can't do WAL ? degrade gracefully
59             return conn
60
61         def _execute_write(self, sql: str, params: Tuple[Any, ...], error_msg: str) -> bool:
62             """Run ONE bool-returning single-statement write inside a connection scope.
63
64             Captures the connect/commit/except-logger.error/return-False boilerplate shared
by

```

10. user (2026-07-04T08:48:56.783Z)

8534077 review: gate concurrency on effective compute path, not the server default (#466)
d56237f fix(cloud-serving): parallelize the drain under cloud_run (#466)
backend/api/job_worker.py | 55 ++++++

```

backend/config/models.py      | 6 +++
backend/main.py               | 2 +
backend/orchestration.py      | 23 ++++++--
tests/test_api_cloud_dispatch.py | 38 ++++++
tests/test_config_deployment.py | 12 +++-
tests/test_drain_concurrency.py | 94 ++++++
7 files changed, 221 insertions(+), 9 deletions(-)

```

11. assistant (2026-07-04T08:49:07.004Z)

The PR added `tests/test\_drain\_concurrency.py`. Let me read it plus the job_worker changes for context.

12. user (2026-07-04T08:49:08.450Z)

```

1      """Drain-executor concurrency (#466).
2
3      The in-process job drain is serial (max_workers=1) for truly-local ? a single box
4      serializes GPU
5      work and the Gemini provider is rate-fragile. Under cloud_run the box only DISPATCHES +
6      bucket-polls
7      (I/O-bound; GPU is on ephemeral L4s, WASB-only has no Gemini), so it scales to
8      ``deployment.max_concurrent_remote_jobs``. The worker count is fixed at first executor
9      creation from
10     the config (``configure_drain``); everything else stays byte-identical to the serial
11     default.
12     """
13
14     import pytest
15     from pydantic import ValidationError
16
17     import backend.api.job_worker as jw
18     from backend.config.models import AppConfig
19
20     def _reset():
21         if jw._job_executor is not None:
22             jw._job_executor.shutdown(wait=False)
23             jw._job_executor = None
24             jw._drain_max_workers = 1
25
26     def test_resolve_drain_workers_serial_by_default():
27         assert jw._resolve_drain_workers(None) == 1 # no config (tests / non-server)
28         assert jw._resolve_drain_workers(AppConfig()) == 1 # default compute_target="" ?
29         serial
30
31     def test_resolve_drain_workers_scales_under_cloud_run(monkeypatch):
32         monkeypatch.delenv("CLOUD_RUN_JOB", raising=False)
33         monkeypatch.delenv("GOOGLE_CLOUD_PROJECT", raising=False)
34         cloud = AppConfig.model_validate(
35             {"deployment": {"compute_target": "cloud_run", "max_concurrent_remote_jobs": 5}}
36         )
37         assert jw._resolve_drain_workers(cloud) == 5
38         # a truly-local (local_gpu) target with no cloud env stays serial even with a high
39         knob
40         local = AppConfig.model_validate(
41             {"deployment": {"compute_target": "local_gpu", "max_concurrent_remote_jobs": 5}}
42         )
43         assert jw._resolve_drain_workers(local) == 1
44
45     def test_resolve_drain_workers_scales_for_cloud_override_on_local_server(monkeypatch):
46         # #468: a local-default server whose per-request compute="cloud" path IS wired (env

```

```

+ an output
45         # bucket) gets the concurrent workers too ? those overrides are the same I/O-bound
remote dispatch.
46         monkeypatch.setenv("CLOUD_RUN_JOB", "rally-worker")
47         monkeypatch.setenv("GOOGLE_CLOUD_PROJECT", "proj")
48         wired = AppConfig.model_validate(
49             {
50                 "deployment": {"compute_target": "", "max_concurrent_remote_jobs": 4},
51                 "storage": {"output_root": "gs://b/out"},
52             }
53         )
54         assert jw._cloud_dispatch_possible(wired) is True
55         assert jw._resolve_drain_workers(wired) == 4
56         # ...but with no output bucket the override path can't actually dispatch ? stay
serial
57         no_bucket = AppConfig.model_validate(
58             {"deployment": {"compute_target": "", "max_concurrent_remote_jobs": 4}}
59         )
60         assert jw._cloud_dispatch_possible(no_bucket) is False
61         assert jw._resolve_drain_workers(no_bucket) == 1
62
63
64     def test_configure_drain_sets_executor_worker_count():
65         _reset()
66         try:
67             jw.configure_drain(
68                 AppConfig.model_validate(
69                     {
70                         "deployment": {
71                             "compute_target": "cloud_run",
72                             "max_concurrent_remote_jobs": 6,
73                         }
74                     }
75                 )
76             )
77             assert jw._get_executor()._max_workers == 6
78         finally:
79             _reset()
80
81
82     def test_executor_serial_without_configure():
83         _reset()
84         try:
85             assert jw._get_executor()._max_workers == 1 # default ? today's serial
behaviour
86         finally:
87             _reset()
88
89
90     def test_max_concurrent_remote_jobs_is_bounded():
91         with pytest.raises(ValidationError):
92             AppConfig.model_validate({"deployment": {"max_concurrent_remote_jobs": 0}}) #
ge=1
93         with pytest.raises(ValidationError):
94             AppConfig.model_validate({"deployment": {"max_concurrent_remote_jobs": 999}}) #
le=64
95

```

13. user (2026-07-04T08:49:09.082Z)

```

1     """Async job worker ? drain/wake loop (DEFERRED_HARDENING_PLAN #16, EXECUTION_POLICY.md
P3).
2
3     Extracted verbatim from backend.main to shrink the orchestrator; behavior is unchanged.
4     The worker DRAINS the jobs table via `claim_due_job` and honours WAIT_AND_RESUME: a job

```

```

5     that raises `Jobwaiting` is parked (status 'waiting' + `next_poll_at` + progress) and
6     re-claimed when due ? with no central scheduler (the DB is the clock).
7
8     The monkeypatch contract these functions rely on (and that tests depend on):
9     every cross-function seam ? `db_instance`, `global_config`, `_execute_processing`,
10    `_arm_wake_timer`, `_get_executor`, `_MAX_DRAIN_WAKE_SEC` ? is resolved THROUGH the
11    `backend.main` module object at call-time (`import backend.main as _main`), never as a
12    bare local. `backend.main` re-exports these names, so `monkeypatch.setattr(backend.main,
13    ...)`` still intercepts the inter-function calls (e.g. a parked job arms the *patched*
14    `_arm_wake_timer`, so no real daemon timer leaks in tests). Importing `backend.main`
    lazily
15    inside the functions also avoids the circular import at module load (main.py imports
    this
16    module; this module must not import main at import time).
17    """
18
19    import json
20    import logging
21    import os
22    import threading
23    import traceback
24    from concurrent.futures import ThreadPoolExecutor
25    from typing import Any, Dict, Optional
26
27    logger = logging.getLogger(__name__)
28
29    # Async job executor (DEFERRED_HARDENING_PLAN #16). max_workers=1 by default: a single
30    # self-hosted box serializes GPU work, and the Gemini provider is rate-fragile under
31    # parallel uploads (2026-06-10 ops learning: serial uploads + backoff). The segmenters
32    # already parallelize their AI handoff internally via their own pools.
33    # EXCEPTION (#466): when the server CAN dispatch to Cloud Run (see
    _cloud_dispatch_possible), a REMOTE
34    # job is only a bucket-poll (I/O-bound; GPU is on ephemeral L4s, WASB-only has no
    Gemini), so the drain
35    # scales to deployment.max_concurrent_remote_jobs to overlap those. This does NOT
    parallelize local
36    # work: CPU/GPU-heavy in-process detection + the compile stitch serialize via
37    # orchestration._LOCAL_COMPUTE_LANE regardless of the worker count, so the single-box
    invariant holds
38    # and compile/local-override jobs stay serial. The count is fixed at first
    _get_executor() creation.
39    #
40    # ? O2 RISK (CODE_CLEANUP_AUDIT $3c): a single worker means ONE stuck or hung job
41    # (e.g. hung Gemini generate, hung WSL GPU call, hung Cloud Run poll) blocks ALL
42    # subsequent jobs ? they stay 'queued' forever. Mitigations already in place:
43    #   - ExecutionPolicy op_timeout_sec kills hung generate/processing calls
44    #   - detector timeout_sec kills hung WSL/GPU calls
45    #   - Cloud Run poll has a max_wait_sec bound
46    #   - ffmpeg/ffprobe subprocesses use bounded media timeouts
47    # A future slice should add a per-job wall-clock timeout (the executor itself
48    # cannot enforce timeouts on the Thread).
49    _job_executor: Optional[ThreadPoolExecutor] = None
50    # Drain worker count, fixed at first _get_executor() creation. 1 (serial) by default;
    configure_drain()
51    # raises it for cloud_run (#466). A module global (not a _get_executor arg) so the many
    call sites +
52    # the tests that monkeypatch `_get_executor` as a no-arg lambda are unchanged.
53    _drain_max_workers: int = 1
54
55
56    def _cloud_dispatch_possible(config: Any = None) -> bool:
57        """True if this server CAN dispatch a job to Cloud Run ? either the profile sets
58        ``deployment.compute_target=cloud_run``, OR the per-request ``compute='cloud'``
    override path is
59        wired (``CLOUD_RUN_JOB`` + ``GOOGLE_CLOUD_PROJECT`` env + an output bucket). Mirrors

```

```

60         orchestration._cloud_available so drain concurrency tracks the ACTUAL remote-
dispatch capability,
61         not just the server default (so compute='cloud' overrides on a local-default server
parallelize too)."""
62         dep = getattr(config, "deployment", None)
63         if getattr(dep, "compute_target", "") == "cloud_run":
64             return True
65         if os.environ.get("CLOUD_RUN_JOB") and os.environ.get("GOOGLE_CLOUD_PROJECT"):
66             stor = getattr(config, "storage", None)
67             if (getattr(stor, "output_root", "") or "").strip():
68                 return True
69         return False
70
71
72     def _resolve_drain_workers(config: Any = None) -> int:
73         """Drain concurrency (#466): a server that CAN dispatch to Cloud Run (see
cloud_dispatch_possible)
74         may run ``deployment.max_concurrent_remote_jobs`` drain workers so I/O-bound remote
dispatch/poll
75         OVERLAPS. This does NOT parallelize CPU/GPU-heavy LOCAL work ? in-process detection
and the compile
76         stitch serialize via orchestration._LOCAL_COMPUTE_LANE regardless of the worker
count ? so the
77         single-box GPU + rate-fragile Gemini invariant holds, and compile/local-override
jobs stay serial.
78         A server with no remote capability stays serial (1). Defaults to 1 when config is
absent
79         (tests / non-server) ? behaviour unchanged."""
80         if _cloud_dispatch_possible(config):
81             dep = getattr(config, "deployment", None)
82             return max(1, int(getattr(dep, "max_concurrent_remote_jobs", 1) or 1))
83         return 1
84
85
86     def configure_drain(config: Any = None) -> None:
87         """Set the drain worker count from ``config`` BEFORE the executor is created (#466).
Call once at
88         server startup. No-op effect once the executor already exists (the count is fixed at
creation)."""
89         global _drain_max_workers
90         _drain_max_workers = _resolve_drain_workers(config)
91
92
93     def _get_executor() -> ThreadPoolExecutor:
94         """Lazy module-global executor; tests monkeypatch this for inline execution. Worker
count is
95         ``_drain_max_workers`` (1 by default; raised for cloud_run via ``configure_drain``,
#466)."""
96         global _job_executor
97         if _job_executor is None:
98             _job_executor = ThreadPoolExecutor(
99                 max_workers=_drain_max_workers, thread_name_prefix="jobworker"
100             )
101         return _job_executor
102
103
104     class JobWaiting(Exception):
105         """Raised inside the pipeline to SELF-SCHEDULE a job (EXECUTION_POLICY.md P3,
WAIT_AND_RESUME). It means "I can't finish now ? park me and resume in `delay_sec`,
106         NOT a failure. The worker persists `next_poll_at` + `progress` (via
`set_job_waiting`),
107         releases the thread, and re-claims the job when due (`claim_due_job`). No work to
date
108         is the segmenter's `_execute_processing`, run UNDER the destroy-after-confirm
contract.

```

```

110
111     The wiring is additive: the production segmenter path never raises this today (zero
112     behaviour change), so a job runs exactly as before. The hook is what a later slice
113     (resumable AI handoff) raises once a long op exhausts its in-process wait budget.
114     """
115
116     def __init__(self, delay_sec: float, progress: Optional[str] = None) -> None:
117         super().__init__(f"job parked for {delay_sec:.0f}s")
118         self.delay_sec = max(0.0, float(delay_sec))
119         self.progress = progress
120

```

14. user (2026-07-04T08:49:50.162Z)

The file C:/Users/avidu/Projects/khelsutra-guru/wt-drain-concurrency/backend/infrastructure/database_jobs.py has been updated successfully. (file state is current in your context ? no need to Read it back)

15. assistant (2026-07-04T08:49:55.739Z)

Now the regression test. I'll add it to `tests/test\_job\_worker\_loop.py` (it already hosts the DB-level `claim\_due\_job` tests).

16. user (2026-07-04T08:49:59.066Z)

The file C:/Users/avidu/Projects/khelsutra-guru/wt-drain-concurrency/tests/test_job_worker_loop.py has been updated successfully. (file state is current in your context ? no need to Read it back)

17. user (2026-07-04T08:49:59.893Z)

The file C:/Users/avidu/Projects/khelsutra-guru/wt-drain-concurrency/tests/test_job_worker_loop.py has been updated successfully. (file state is current in your context ? no need to Read it back)

18. user (2026-07-04T08:50:14.563Z)

The file C:/Users/avidu/Projects/khelsutra-guru/wt-drain-concurrency/tests/test_job_worker_loop.py has been updated successfully. (file state is current in your context ? no need to Read it back)

19. assistant (2026-07-04T08:50:20.193Z)

Now run the new test repeatedly to check determinism under thread pressure (SQLite locking), before the full suite.

20. user (2026-07-04T08:50:28.623Z)

```

..... [100%]
16 passed in 3.93s

```

21. user (2026-07-04T08:50:39.866Z)

```

inifile: C:\Users\avidu\Projects\khelsutra-guru\wt-drain-concurrency\pytest.ini
rootdir: C:\Users\avidu\Projects\khelsutra-guru\wt-drain-concurrency

. [100%]
1 passed in 2.22s

```

22. assistant (2026-07-04T08:50:44.715Z)

pytest-repeat isn't installed; loop it manually.

23. user (2026-07-04T08:51:43.919Z)

failures: 0 / 15

24. assistant (2026-07-04T08:51:53.016Z)

The test passes consistently. Now let me verify the test actually catches the pre-fix bug ? temporarily stash the fix and run the test against the old code.

25. user (2026-07-04T08:52:18.684Z)

pre-fix failures caught: 5 / 5

M backend/infrastructure/database_jobs.py

M tests/test_job_worker_loop.py

26. assistant (2026-07-04T08:52:24.845Z)

The test caught the pre-fix bug 5/5 times, and the fix is restored. Now the three gates.

27. user (2026-07-04T08:54:24.234Z)

```
tests/test_api_endpoints.py::test_process_bucket_uri_yields_queryable_video_id[gcs://bucket/videos/x.mp4]
```

```
C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\site-packages\google\auth\_default.py:113: UserWarning: Your application has authenticated using end user credentials from Google Cloud SDK without a quota project. You might receive a "quota exceeded" or "API not enabled" error. See the following page for troubleshooting: https://cloud.google.com/docs/authentication/adc-troubleshooting/user-creds.
warnings.warn(_CLOUD_SDK_CREDENTIALS_WARNING)
```

-- Docs: <https://docs.pytest.org/en/stable/how-to/capture-warnings.html>

1634 passed, 10 skipped, 8 warnings in 112.63s (0:01:52)

28. user (2026-07-04T08:54:35.153Z)

ruff 0.15.16

All checks passed!

mypy 2.1.0 (compiled: yes)

backend\annotations\base.py:88: note: By default the bodies of untyped functions are not checked, consider using --check-untyped-defs [annotation-unchecked]

backend\sports__init__.py:9: note: By default the bodies of untyped functions are not checked, consider using --check-untyped-defs [annotation-unchecked]

backend\pipeline\validators\base.py:44: note: By default the bodies of untyped functions are not checked, consider using --check-untyped-defs [annotation-unchecked]

backend\providers__init__.py:9: note: By default the bodies of untyped functions are not checked, consider using --check-untyped-defs [annotation-unchecked]

Success: no issues found in 147 source files

29. assistant (2026-07-04T08:54:41.188Z)

All gates green. Final diff check.

30. user (2026-07-04T08:54:44.139Z)

backend/infrastructure/database_jobs.py | 45 ++++++

tests/test_job_worker_loop.py | 43 ++++++

2 files changed, 71 insertions(+), 17 deletions(-)

8534077 review: gate concurrency on effective compute path, not the server default (#466)

31. user (2026-07-04T08:55:05.766Z)

Structured output provided successfully